

Back to Basic: The Shell

Before setting up any Big Data Systems

06026213 Big Data Systems

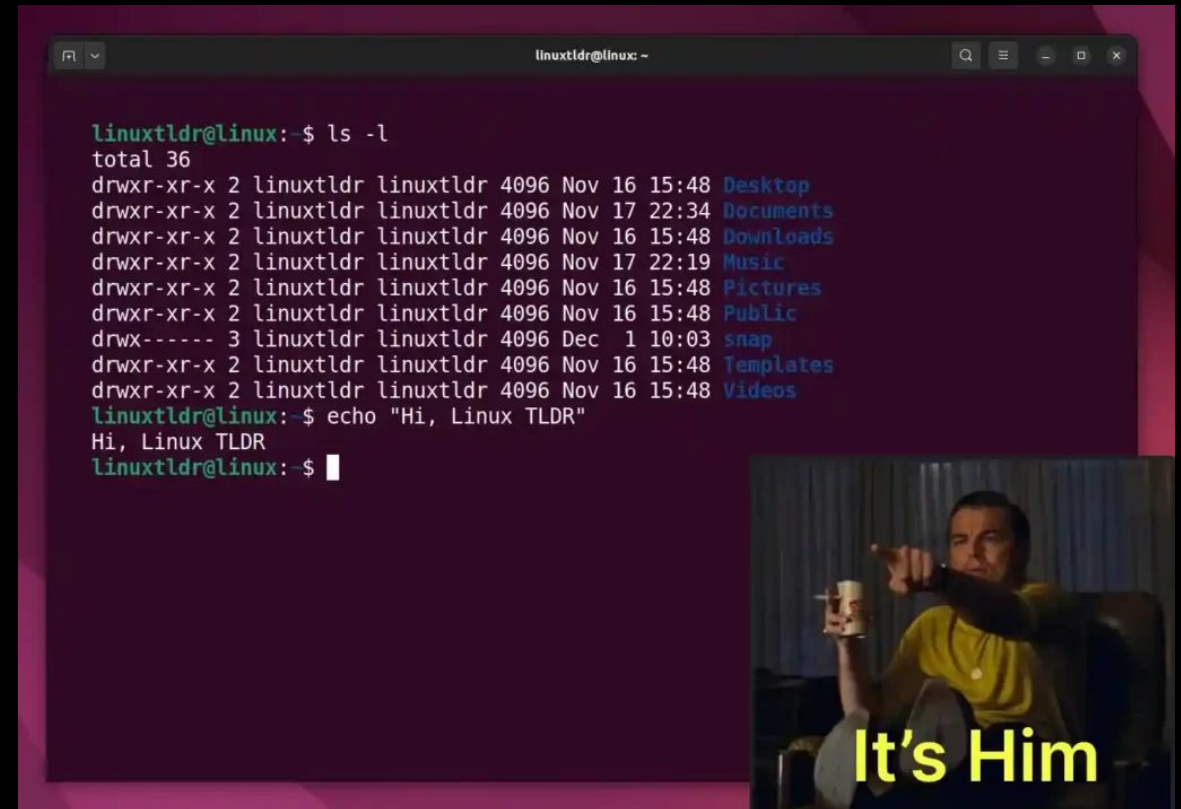
Dr. Sirasit Lochanachit

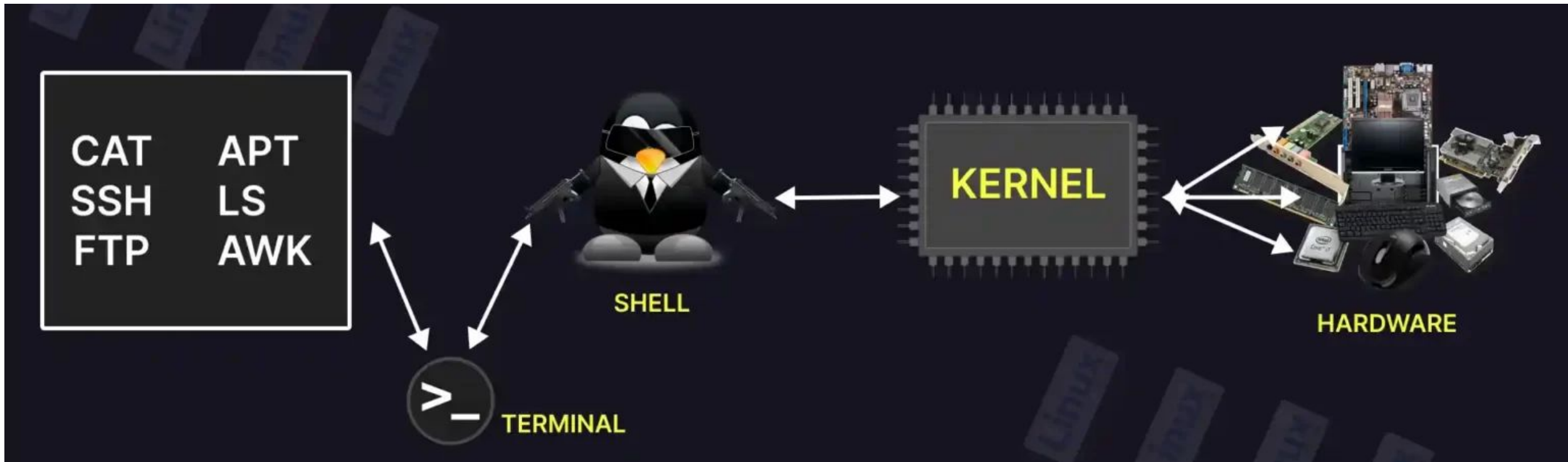
What is The Shell?

Modern fanciful GUIs, voice interfaces, and LLMs are great for 80% of use-cases, but they are fundamentally restricted; you can't press a button that isn't there.

The Shell is a **textual interface** that allows you to run programs, give input, and inspect output in a semi-structured way.

- Combines programs creatively
- Automates nearly any task
- The core of open-source software





The shell is a program that carries the user typed commands or instructions from the terminal and converts them into something that the kernel can understand.

The kernel is a standard program that resides in system memory in major systems to create a bridge between the hardware and the software.

Accessing the Shell via Your Terminal

A terminal is the visual interface used to access the shell.

Platform	Access Method	Primary Shell
Linux 	Press Ctrl + Alt + T	bash (Bourne Again Shell)
macOS 	Cmd + Space, type "Terminal"	bash / zsh
Windows 	Win + R, type "powershell" or "cmd"	PowerShell (use WSL for bash)

Recommendation: For this class, Windows users should use WSL (Windows Subsystem for Linux).

Navigating in The Shell?

When a terminal is launched, a prompt will show up.

The Prompt usually looks like `bds@bds-itkmitl:~$`

- The `~` represents your home directory, and `$` indicates you are a regular user (not root).

Try enter 'date' command

Then enter a command with arguments: 'echo hello'

```
bds@bds-it-kmitl:~$ _
```

Navigating the File System in the Shell



`pwd`

Print Working Directory.

Displays exactly where you are in the file system.



`cd`

Change Directory. Pass a path (absolute or relative) to change your working location.



`ls`

List Content. Shows files and directories in the current or specified path.

Special Paths:

- `.` (This directory)
- `..` (Parent directory)
- `~` (Home directory)

Absolute paths start with `/`, relative paths do not.

How the Shell Find Programs

\$PATH

The Environment Variable

When you run a command, the shell searches through a colon-separated list of directories stored in \$PATH to find the executable file/program.

- \$PATH is an environment variable

Use `which` (e.g., `which echo`) to find the exact file path of any program being executed.

The Data Science Toolbox



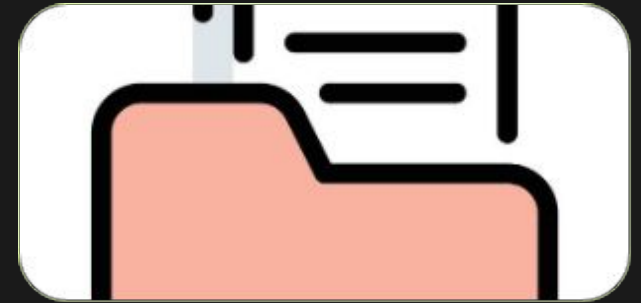
grep

Finds lines matching a regular expression pattern in files.
Essential for log analysis.



awk / sed

Powerful parsing and editing tools. awk is ideal for columnar data (CSV/TSV).



find

Recursively find files matching specific conditions like name, size, or time.

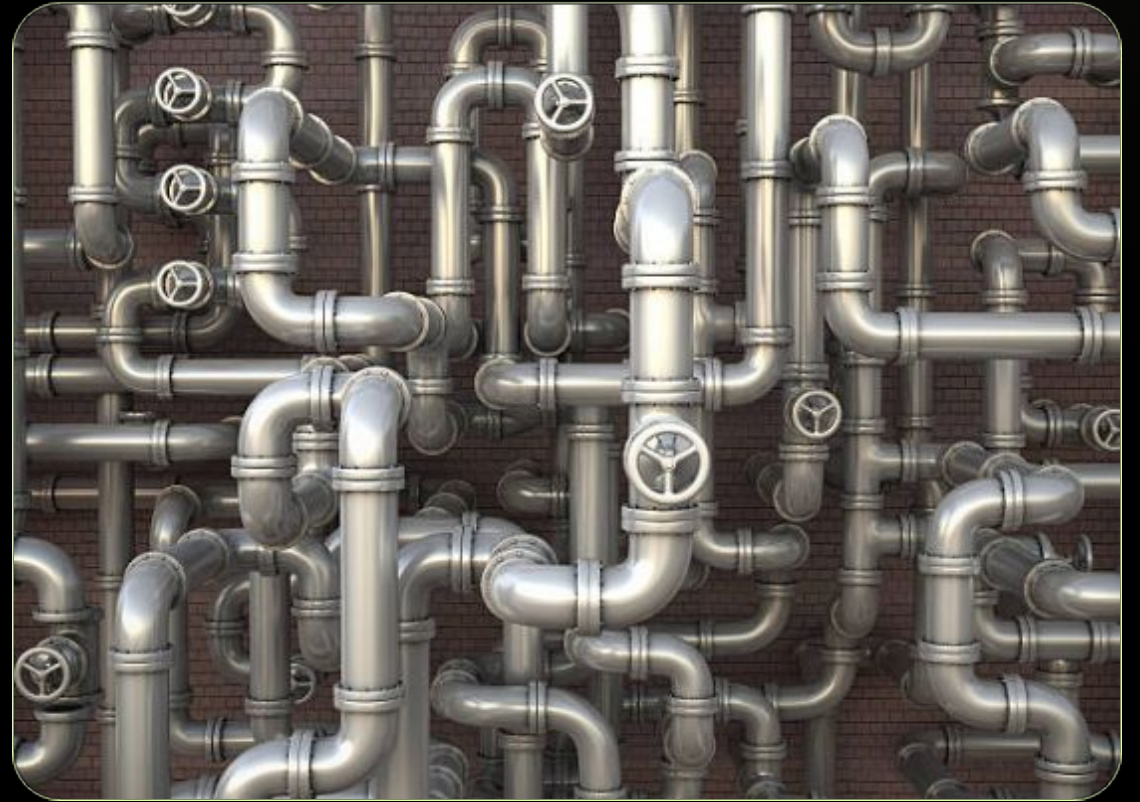
Composition with Pipes

Connect Programs

The `|` pipe takes the "standard output" of one program and makes it the "standard input" of the next.

```
grep "Error" log.txt | awk '{print $1}' | sort | uniq
```

This allows you to **compose** simple tools into complex, powerful pipelines.



| Stream Redirects

Output (> & >>)

> file: Takes standard output and writes it to a file (overwrites)

>> file: Appends to a file rather than overwriting it.

Input (<)

< file: Tells the shell to read from a file instead of from your keyboard.

This provides standard input to a program from an existing file.

Exercises

1. Try `ls -l /` and examine the output. What does it do?
2. Write a command that copies a file to a backup with today's date in the filename (e.g., `notes.txt` → `notes_2026-01-12.txt`). (Hint: `$(date +%Y-%m-%d)`)
3. Use pipes to find the 5 most common file extensions in your home directory. (Hint: combine `find`, `grep`, `sort`, `uniq`, and `head`.)